

CSE4502 (Operating Systems Lab): 1A

Jibon Naher¹ and Samnun Azfar²

¹Assistant Professor, CSE

²Junior Lecturer, CSE

February 17, 2026

Instruction

- Use the PC from the lab
- Total time is 40 minutes
- Submit your phone at the front
- Turn off the internet connection

1 Task: Preemptive SJF Scheduling Algorithm

1.1 Given Code (Do NOT modify this part)

The following structure and main function are provided. Your implementation must integrate seamlessly with this boilerplate.

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <iomanip>
5
6 using namespace std;
7
8 struct Process {
9     int id;
10    int at; // Arrival Time
11    int bt; // Burst Time
12    int ct; // Completion Time
13    int tat; // Turnaround Time
14    int wt; // Waiting Time
15    int remaining; // Remaining Time for preemption
16 };
17
18 // --- Your Implementation Goes Here ---
19 void preemptiveSJF(vector<Process>& processes) {
20     // Implement the SRTF logic here
21 }
22
23 int main() {
24     vector<Process> processes = {
25         {1, 0, 6, 0, 0, 0, 6},
26         {2, 2, 4, 0, 0, 0, 4},
27         {3, 4, 2, 0, 0, 0, 2},
28         {4, 6, 5, 0, 0, 0, 5}
29     };
30 }
```

```

31 preemptiveSJF(processes);
32 return 0;
33 }

```

1.2 Implementation Hints

To successfully implement the `preemptiveSJF` function, follow this time-unit simulation approach:

- **Initialize Time:** Start a system clock (e.g., `currentTime = 0`) and track completed processes.
- **At each time unit (increment by 1):**
 1. Find all processes that have arrived (`at <= currentTime`) and are not yet finished (`remaining > 0`).
 2. From these available processes, select the one with the **shortest remaining time (rt)**.
 3. **Tie-breaking:** If multiple processes have the same `rt`, pick the one with the earliest arrival time (`at`).
 4. Execute the selected process for **1 time unit** (decrement its remaining time and increment `currentTime`).
 5. **Completion Check:** If a process's `rt` becomes 0:
 - Mark it as completed.
 - Record its `CT` (current time).
 - Calculate $TAT = CT - AT$ and $WT = TAT - BT$.
 6. **Idle Handling:** If no process has arrived yet, increment the clock until the next arrival.

1.3 Your Task

Complete the `preemptiveSJF` function to fulfill the following requirements:

1. Simulate SRTF using the logic described above.
2. Print a table showing the ID, AT, BT, CT, TAT, and WT for every process.
3. Compute and display the **Average Turnaround Time** and **Average Waiting Time**.

CSE4502 (Operating Systems Lab): 1A

Jibon Naher¹ and Samnun Azfar²

¹Lecturer, CSE

²Junior Lecturer, CSE

January 19, 2026

1 Part A: File Guard (Conditionals)

1.1 Problem Statement

Develop a Bash script named `file_guard.sh` that verifies whether a specified file exists and determine whether the provided file name corresponds to a regular file, a directory, or other.

1.2 Logic

1. Take a filename as a command-line argument.
2. Use `if [ -e $filename ]` to check existence.
3. If it exists, check if it is a directory (`-d`) or a file (`-f`).

1.3 Hints

- Use `$1` to access the first argument passed to the script.

1.4 Example Output

Case 1: File exists

```
$ ./file_guard.sh notes.txt  
'notes.txt' is a regular file.
```

Case 2: Directory exists

```
$ ./file_guard.sh Documents  
'Documents' is a directory.
```

Case 3: File does not exist

```
$ ./file_guard.sh unknown.txt  
Error: 'unknown.txt' does not exist.
```

2 Part B : Thread-Safe Histogram

2.1 Problem Statement

A large dataset is processed by multiple threads. They update a shared frequency array (histogram) of size K .

```
histogram[10] = {0, 0, ...}
```

Rules

- Input is an array of n integers which have values k ranging from $0 \leq k < K$. k is an integer.
- Assume you have $n = 9$. If you create 2 threads then thread t_1 will calculate the histogram for indices $0 - 4$ and thread t_2 will calculate the histogram for indices $5 - 8$, both done in a histogram array local to their threads. Then both the threads concurrently update the global histogram using the locally generated histograms.

Inputs & Outputs

- Input:** First line of input contains n <space> K .
- Output:** Histogram[i]: count for i in $0..(K - 1)$.

2.4 Constraints

- Use one mutex.

2.5 Invariant to Preserve

$$\sum \text{histogram}[i] = n$$